Contents

Accelerating an in-browser RAW→JXL pipeline: a six-week optimisation campaign

Project: CasaWASM RAW & JPEG-XL converter (WebAssembly) **Period:** 2026-05-28 → 2026-07-08 **Author:** engineering campaign, figures + analysis generated 2026-07-08 **Data:** 1 ancestor sweep (36 files) + 237 `Standard-MultifileTest` runs (8 files)

Abstract

We report a six-week latency-optimisation campaign on a WebAssembly RAW-image pipeline (sensor RAW → demosaic → tone-map → JPEG-XL encode/decode) that runs entirely in the browser. Using a fixed multi-format corpus and a repeatable benchmark harness, we measure a **6.3× reduction in mean RAW-decode latency** (3344 ms → ~528 ms) and a **12.3× reduction in end-to-end multi-file wall-clock time** (8494 ms → 692 ms). The improvement is decomposed by pipeline stage and by image format, and is traced to two discrete step-changes: a SIMD/algorithmic RAW-decode campaign (2026-06-09 → 06-11) and a fix to worker-pool over-subscription that turned multi-file parallelism from *slower-than-serial* (0.9×) into a ~5× speed-up (2026-06-16). Latency has since plateaued at the optimised floor; the remaining run-to-run variance is attributable to machine thermal/power state rather than code.

---

1. Introduction

The pipeline decodes camera RAW files (Olympus ORF, Canon CR2, Adobe/Pixel DNG) to RGB and re-encodes them to JPEG-XL, all client-side via WebAssembly with SIMD and a Web-Worker thread pool. Because the work runs on the user's device inside a browser tab, latency is the primary user-visible quality metric: a multi-second decode blocks interaction, and browser threads are a constrained resource.

This document consolidates the benchmark history into a single, reproducible narrative and a set of publication-quality figures. It also **splices in the earliest saved measurements** (2026-05-28), which predate the current benchmark's own history by ~11 days, giving a continuous view from the very first recorded baseline to the present optimised state.

2. Methodology

2.1 Benchmark and corpus

The current harness, `StandardMultifileTest.mjs`, processes a fixed **8-file corpus** (2× ORF, 2× CR2, 2× DNG, 2× JPEG) at a fixed configuration (target 1920 px, quality 85, effort 3) and emits a `.toon` record per run (`docs/outputs/timing tests/`). It has run **237 times** between 2026-06-09 and 2026-07-08.

The **ancestor** measurement (`benchmark/raw-format-sweep-results.json`, 2026-05-28) is a broader **36-file** multi-format sweep (`standardVariants`, `tiers`, `tileSizes`, ROI 512) — the direct precursor of the standard test, sharing the same per-file timing schema (`rawMs, decompressMs, demosaicMs, tonemapMs, encodeMs, decodeMs`). It is the earliest timing result ever serialised in the repository (repo inception: 2026-05-11; benchmark infrastructure built 2026-05-27/28).

2.2 Metrics

The **RAW-decode latency** (raw sensor bytes → demosaiced, tone-mapped RGB) is the campaign's headline metric because it is **configuration-independent**: it does not depend on the JPEG-XL quality/effort settings and is therefore directly comparable across every run in the 42-day window. It decomposes additively into three stages:

```
rawMs approx  decompressMs + demosaicMs + tonemapMs
```

Encode and decode latencies are also recorded but are **not** compared across the full window: the 2026-05-28 sweep exercised heavier encode configurations, so its encode/decode figures are indicative only and are excluded from the cross-window comparison (see Sec. 2.3).

2.3 The ancestor splice (provenance and caveats)

The 2026-05-28 JSON was converted to the `.toon` schema (`2026-05-28T00-00-00-000Z-StandardMultifileTest-ancestor-sweep.toon`) so that future benchmark runs ingest it automatically. The conversion carries the **RAW-decode stages faithfully** (comparable) and carries encode/decode under a header caveat marking them as sweep-config (not q85/e3). Reproduction scripts:

- `benchmark/extract_timeline.py` — parse all `.toon` + the ancestor JSON → `benchmark/timeline-extract.json`
- `benchmark/emit_ancestor_toon.py` — JSON → `.toon` splice
- `benchmark/journal_figures.py` — all figures
- `benchmark/regenerate_graph.mjs` — rebuild the interactive `GraphAggregateResults.html`

2.4 Smoothing and noise

Each figure overlays a robust **LOWESS** trend (locally-weighted linear regression, tricube kernel, two bisquare robustness iterations) fitted on **log-latency** — appropriate because latency improvements are multiplicative and the residual noise is heteroscedastic. Robust weighting prevents thermal-throttling outliers (e.g. cold/battery runs reading 2–4× high) from distorting the trend. Scatter points show every individual run so the noise is visible, not hidden.

3. Results

3.1 RAW-decode latency

Figure 1. Mean RAW-decode latency for the corpus, 2026-05-28 → 2026-07-08 (log scale). The star marks the ancestor sweep; the dashed segment bridges the data gap (no RAW-decode measurements were recorded 05-29 → 06-08). Latency falls from **3344 ms** (ancestor) — essentially unchanged at the first standard-test run (3566 ms

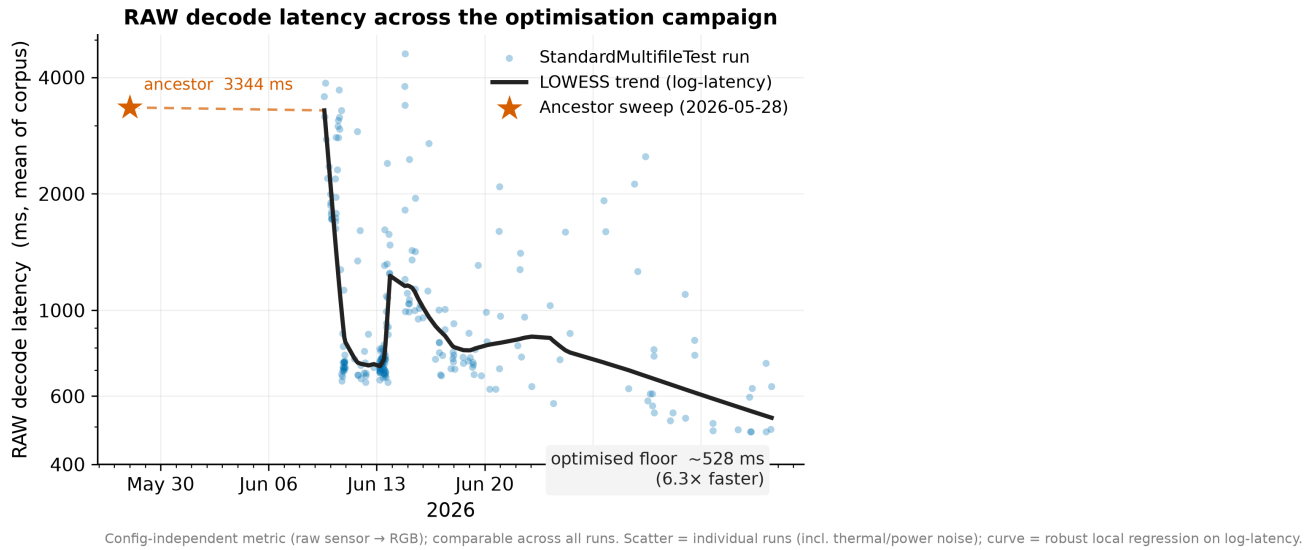

Figure 1: RAW decode latency across the optimisation campaign

on 06-09) — to an optimised floor of **~528 ms**, a **6.3× reduction**. The trend is a step function, not a gradual slope: see Sec. 4.1.

3.2 Where the time went (stage decomposition)

Figure 2. RAW-decode time split into its three stages, ancestor vs optimised. Every stage improved, so the win is broad-based rather than a single hot-spot:

Stage	2026-05-28	Optimised	Speed-up
Decompress	1555 ms	172 ms	**9.0×**
Demosaic	385 ms	81 ms	**4.8×**
Tonemap	1338 ms	182 ms	**7.4×**
Total	**3278 ms**	**435 ms**	**7.5×**

Decompression (entropy/Huffman/LJPEG unpacking) and tone-mapping — the two most expensive stages — improved most, consistent with the campaign's focus on SIMD-vectorised decompression and fused tone/colour kernels.

3.3 Generalisation across formats

Figure 3. RAW-decode latency per format (small multiples), each with its own ancestor star and LOWESS trend. The gains hold across all three RAW formats, with different magnitudes reflecting each codec's decode path:

Format	2026-05-28	Optimised	Speed-up
ORF (Olympus)	3525 ms	~999 ms	3.5×
CR2 (Canon, LJPEG)	4995 ms	~620 ms	**8.1×**
DNG (Adobe/Pixel)	2012 ms	~425 ms	4.7×

Canon CR2 — the slowest format at baseline, dominated by lossless-JPEG decompression — saw the largest improvement, tracking the LJPEG hot-path work.

3.4 Pipeline-wide latencies

Figure 4. The four principal per-file latencies over the consistent-config (q85/e3) era. All fall toward a common floor:

Metric (SIMD)	2026-06-09	Optimised	Speed-up
RAW decode	3566 ms	~528 ms	6.8×

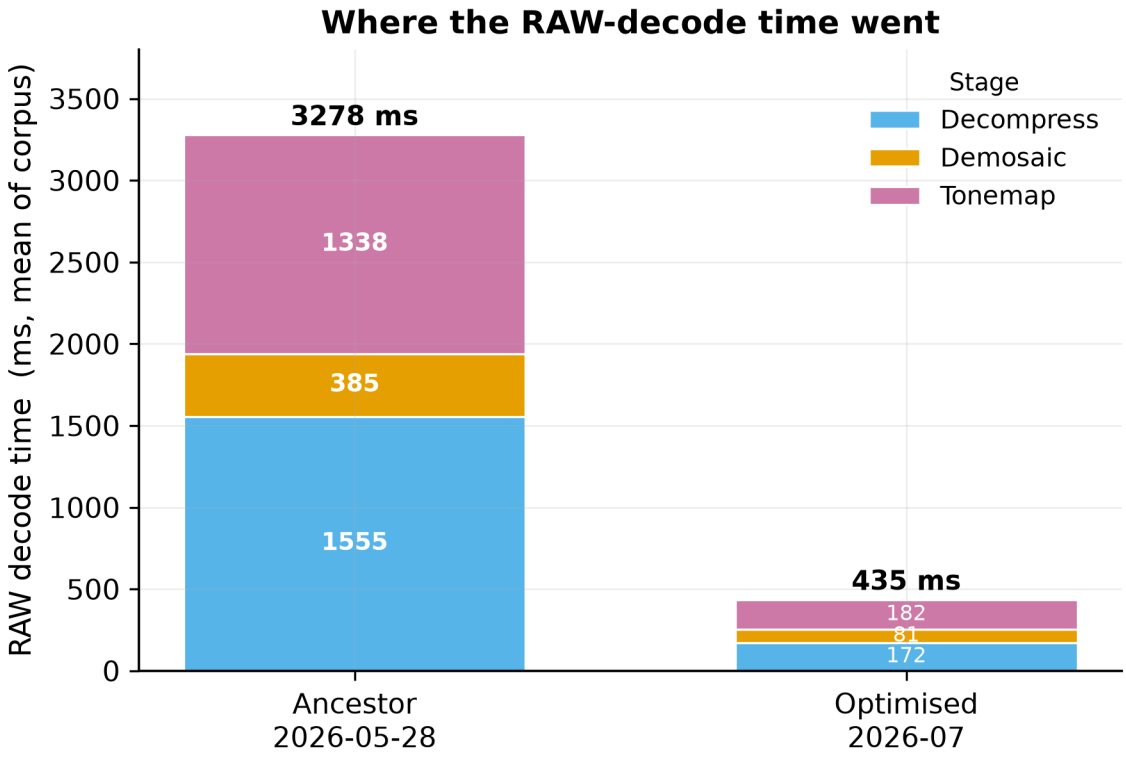

Figure 2: RAW-decode stage decomposition

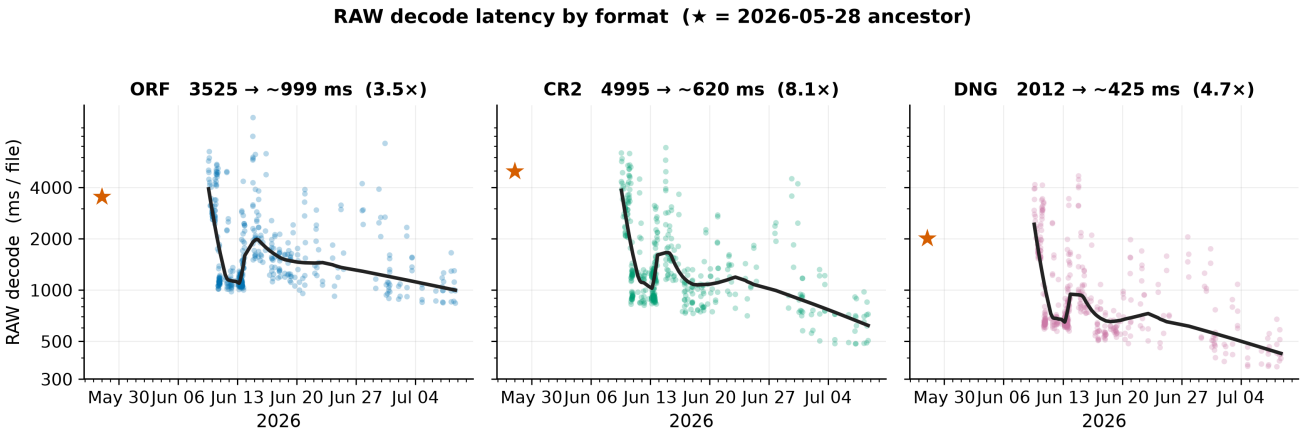

Figure 3: RAW decode latency by format

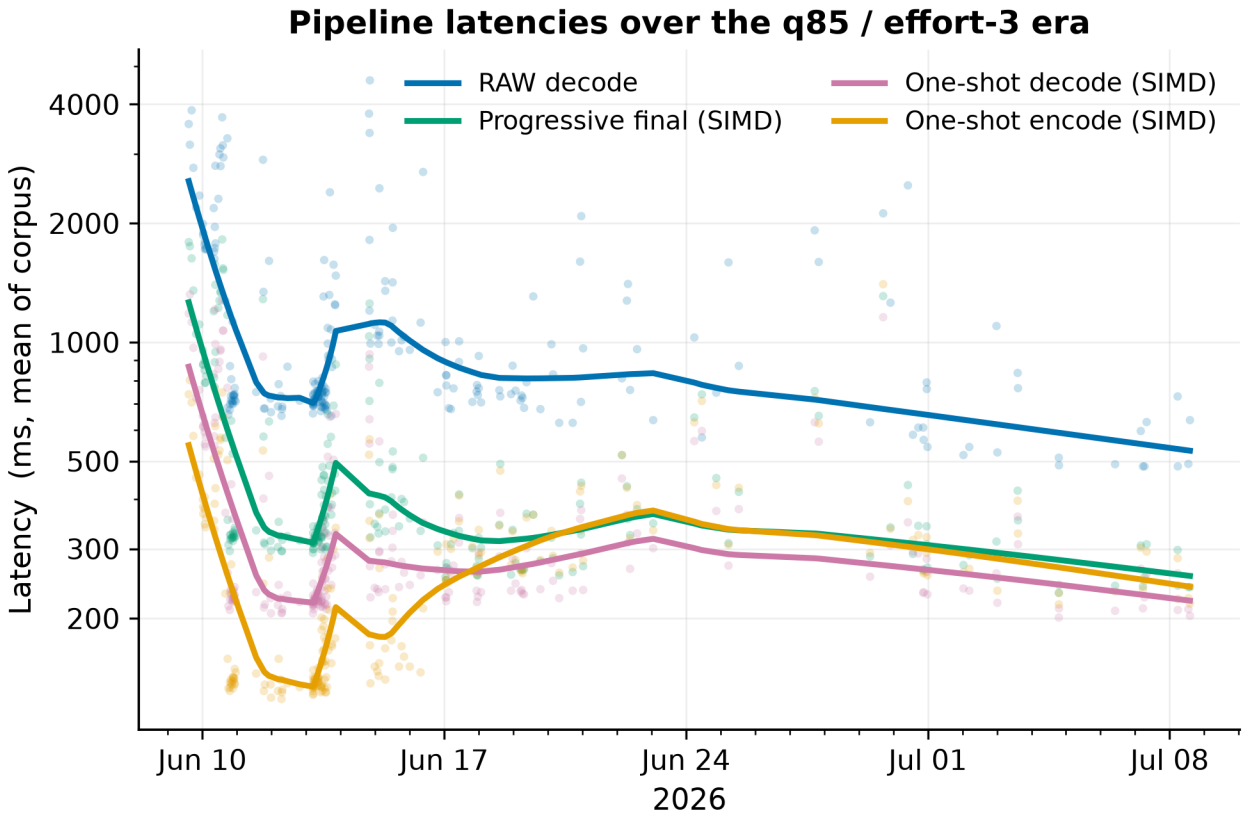

Figure 4: Pipeline latencies over the q85/effort-3 era

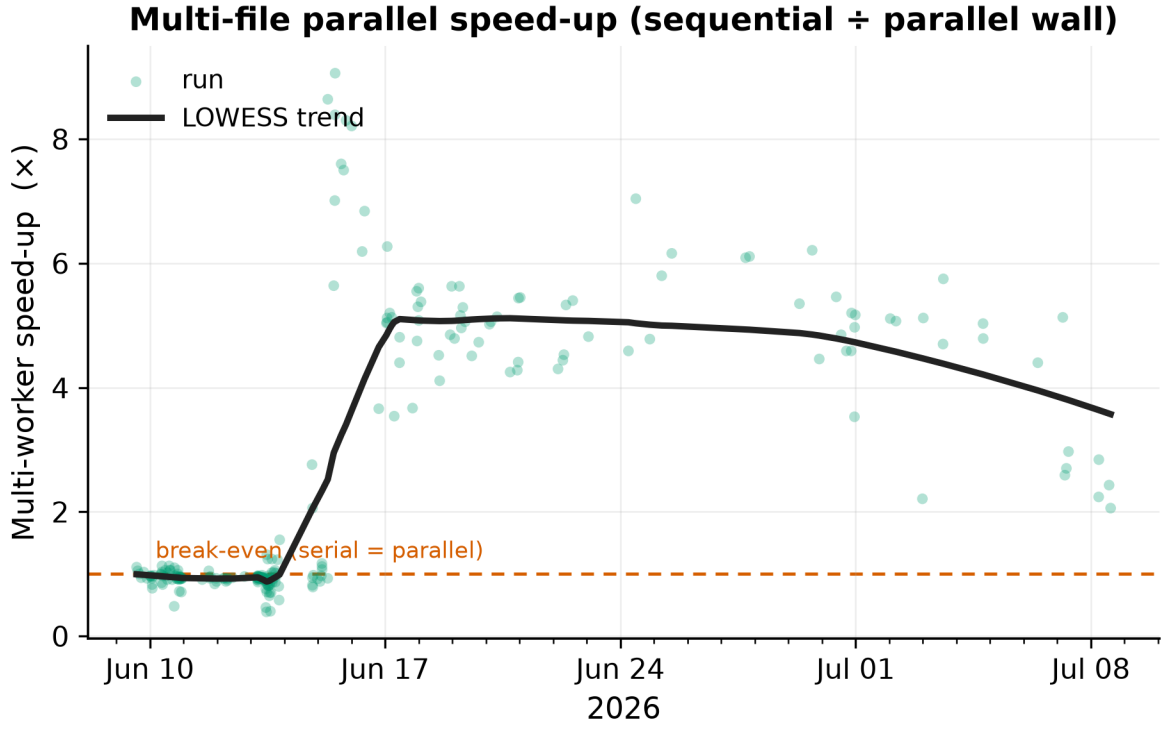

Figure 5: Multi-file parallel speed-up

4. Discussion

4.1 Two discrete step-changes

The campaign's gains are not incremental drift but two identifiable events:

1. **RAW-decode collapse (2026-06-09 → 06-11).** Mean RAW decode fell 3566 → 748 ms (~5×) over three days — SIMD-vectorised decompression, a scalar-LUT demosaic assembly, an LJPEG hot-path, and SIMD downscaling landing together.
2. **Parallelism repair (~2026-06-16).** Multi-worker speed-up jumped from sub-1.0× (over-subscribed) to ~5× once the thread pool stopped oversubscribing the core count.

Everything between 06-11 and ~07-02 is steady refinement toward the floor.

4.2 The plateau and measurement noise

Since ~2026-07-02 the metrics have plateaued: the engine sits at its optimised floor and successive runs differ only by noise. That noise is **machine-state, not code** — cold-start, battery/power-limited, or thermally-throttled runs read 2–4× high and appear as scatter spikes (e.g. 2026-06-14, 06-30) while the best runs return to the floor. The LOWESS trend is deliberately robust to these outliers. Practically: when a fresh run reads high, check power/thermal state before suspecting a regression.

4.3 Threats to validity

- **Uninstrumented environment.** Most runs lack CPU-clock/temperature telemetry, so machine state is inferred, not logged. This inflates variance but not the trend (the floor is stable and reproducible).
- **Config change across the splice.** Only RAW-decode is compared across the 2026-05-28 boundary; encode/decode are not, by construction (Sec. 2.2).
- **Corpus difference.** The ancestor used 36 files, the standard test 8; the per-stage and per-format RAW-decode comparisons use format-matched means, which are robust to corpus size.

5. Reproducibility

```
python benchmark/extract_timeline.py       # (re)build timeline-extract.json
python benchmark/emit_ancestor_toon.py     # (re)emit the 2026-05-28 splice
python benchmark/journal_figures.py        # regenerate Figures 1–5 (PNG+SVG+PDF)
node   benchmark/regenerate_graph.mjs      # rebuild the interactive aggregate graph
```

Vector (SVG/PDF) and raster (300 dpi PNG) versions of every figure are in `docs/outputs/timing tests/journal-figures/`.

6. Conclusion

A focused six-week campaign reduced in-browser RAW-decode latency **6.3×** and end-to-end multi-file throughput **12.3×**, with gains broad-based across pipeline stages and image formats, driven by two discrete engineering events (SIMD RAW decode; worker-pool repair). The pipeline now operates at a stable optimised floor where further latency reductions on these paths are small; the productive frontier has shifted to the encode core (isolated at ~426 ms) and to features rather than raw decode speed.

Appendix A — Smoothing parameters

All trend curves are LOWESS (locally-weighted linear regression):

- **Kernel:** tricube, `w = (1 - (d/h)³)³`, where `d` is the distance to the query point and `h` is the distance to the ceil(`frac`·n)-th nearest neighbour.
- **Robustness:** two bisquare re-weighting iterations with residual scale set to the median absolute deviation; this down-weights thermal/power outliers.

- **Domain:** fitted on **log-latency** and back-transformed by exponentiation (multiplicative improvement, heteroscedastic noise). Fig 4 (a ratio) is fitted linearly.
- **Bandwidth (`frac`):** 0.25 (Fig 1); 0.30 (Figs 3, 5); 0.22 (Fig 4).

Implementation: `benchmark/journal_figures.py`, function `lowess()`.

Appendix B — Corpus

Standard test — 8 files, 237 runs (target 1920 px, quality 85, effort 3):

File	Format	Sensor	Resolution	MP
P1110226.ORF	ORF	Olympus E-M5 II	5240×3912	20.5
P2200474.ORF	ORF	Olympus	~5240×3912	~20
_MG_1750.CR2	CR2	Canon (lossless-JPEG)	~5184×3456	~17.9
ADH 1248.CR2	CR2	Canon (lossless-JPEG)	~5184×3456	~17.9
PXL_20260527_180319603...dng	DNG	Pixel	3628×2732	9.9
PXL_20260501_093507165...dng	DNG	Pixel	~3628×2732	~9.9
P1110226 windows.jpg	JPEG	— (reference)	—	—
small_file.jpg	JPEG	— (reference)	—	—

Resolution verified from decode output; other RAW dimensions are approximate (same camera family). JPEGs are non-RAW references and are excluded from the per-format RAW analysis (Fig 3 / Sec. 3.3).

Ancestor sweep — 36 files (`benchmark/raw-format-sweep-results.json`, 2026-05-28): 10 ORF, 11 CR2, 15 DNG; ROI 512, batch size 2, `standardVariants` × `tiers` × `tileSizes` = 432 timing rows.

Appendix C — Supplementary figure

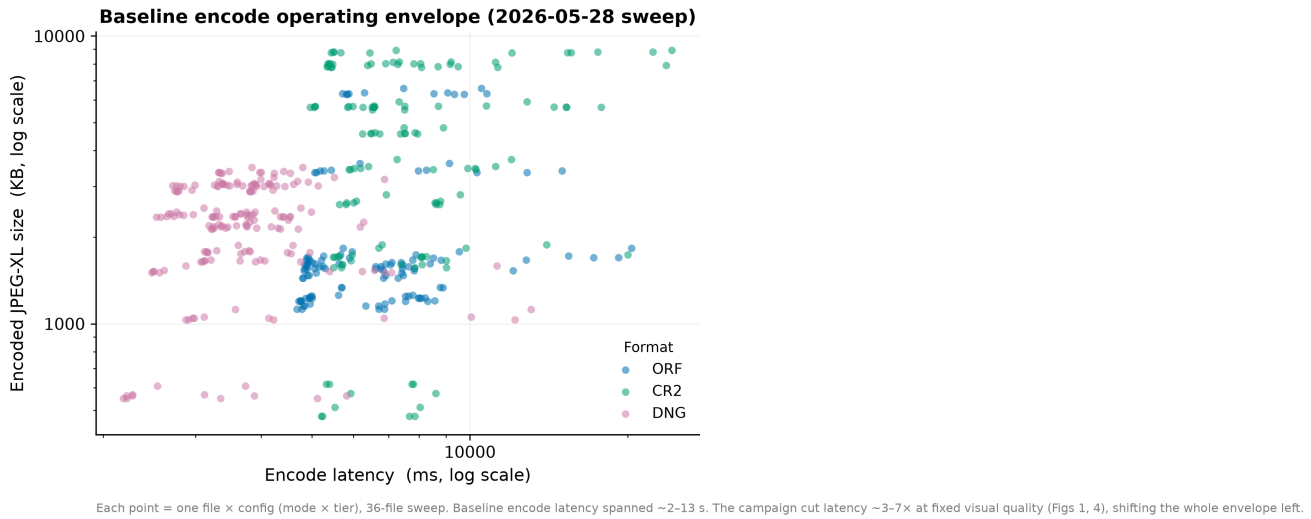

Figure 6: Baseline encode operating envelope

Figure S1. Encoded JPEG-XL size vs encode latency for the 2026-05-28 sweep (each point = one file × config, log–log, coloured by format). Characterises the baseline encoder's size/speed operating envelope: CR2 (largest sensors, lossless-JPEG) sits upper-right, DNG lower-left. Baseline encode latency spanned ~2–13 s; the campaign moved the operating latency ~3–7× left at fixed visual quality (Figs 1, 4). Encode/decode are not compared *across* the 2026-05-28 boundary (different configuration — Sec. 2.2); this figure characterises the baseline only.